



Category Theory in the Scott1980 Lean Formalization

Category theory appears in this repository mainly as Scott's Lecture VI vocabulary and a few later functor/monad exercises — not as a general Mathlib category-theory development.

Design choice: bespoke, not Mathlib

The core abstraction lives in `Scott1980/Neighborhood/Definition63.lean`. It defines a small self-contained layer:

- **Category** — objects, `Hom`, `id`, `comp`, the three laws
- **Endofunctor** — `obj`, `map`, identity/composition preservation
- **TAlgebra**, **AlgHom**, **IsInitial**
- **Iso** (for Lambek-style results)

The file explicitly explains why **Mathlib's CategoryTheory is not imported**: Mathlib's initial-algebra API would pull in `Classical.choice`, while this project's Propositions 6.6–6.7 are proved choice-free. So the repo uses "a small amount of category theory" in Scott's sense, implemented locally.

1. The category of domains (Scott's running example)

Role	Lean type	Scott source
Objects	<code>DomainObj</code> (token type + <code>NeighborhoodSystem</code>)	Lecture VI
Morphisms	<code>ApproximableMap</code>	Def 2.1
Laws	<code>idMap_comp</code> , <code>comp_idMap</code> , <code>comp_assoc</code> (Theorem 2.5)	Thm 2.5

This is the concrete instance of Scott's remark that neighbourhood systems and approximable maps "form quite an interesting category."

2. Lecture VI: endofunctors, algebras, initial algebras

This is the main categorical spine, matching Scott's Definitions 6.3–6.5:

Concept	Lean module	Scott source
Endofunctor	<code>Endofunctor DomainObj</code>	Def 6.3
T -algebra + homomorphisms	<code>TAlgebra</code> , <code>AlgHom</code>	Def 6.4
Initial algebra	<code>IsInitial</code>	Def 6.5
Unique iso of initials	<code>Proposition66.lean</code>	Prop 6.6
Lambek's lemma	<code>lambek</code> in <code>Proposition67.lean</code>	Prop 6.7
Continuous on maps	<code>ContinuousOnMaps</code> in <code>Definition68.lean</code>	Def 6.8
Homomorphisms from fixed point	<code>Theorem69.lean</code>	Thm 6.9
Monotone/continuous on domains	<code>Definition613.lean</code>	Def 6.13
Existence of initial algebra	<code>Theorem614.lean</code> (<code>colimit $\bigcup_n T^n(\{\Gamma\})$</code>)	Thm 6.14
Initial algebra embeds in every solution	<code>Theorem616.lean</code>	Thm 6.16

Theorem 6.14 is explicitly a **colimit** construction: iterate a functor from a generating system Γ and take the union.

3. Cartesian closed category (Exercise 3.23)

`Exercise323.lean` packages Scott's "for category theorists" remark:

- **Terminal object:** `unitSys` (Exercise 3.15)
- **Products:** `prod` , `proj0` , `proj1`

- **Exponentials:** `curryEquiv` from Theorem 3.12

So the category of domains and approximable maps is shown to be **cartesian closed**, with $(\Diamond_0 \rightarrow -)$ as the right adjoint to $- \times \Diamond_0$.

4. Functorial structure on domain constructors

Several files treat `×`, `+`, and `→` as **functors on maps**:

Functor	Module(s)	Scott source
Product <code>prodMap</code>	<code>Exercise319.lean</code> , <code>Exercise320.lean</code>	Exercises 3.19–3.20
Sum <code>sumMap</code>	<code>Exercise319Sum.lean</code>	Exercise 3.19 (sum)
Exponential <code>expMap</code>	<code>FunctionSpace.lean</code> , <code>Proposition810b.lean</code> , <code>Exercise819.lean</code>	Thm 3.12, Ex 8.19

These are used heavily in Lecture VIII (Definition 8.9's combinators on $\Diamond$, Proposition 8.10's projection-closure).

5. Power domain as functor and monad (Lecture VII)

- **Exercise 7.19** (`Exercise719.lean`): $D \mapsto \mathbb{P}D$ is a functor via `Pfmap` / `Pf`, with identity and composition laws.
- **Exercise 7.20** (`Exercise720.lean`): `union` : $\mathbb{P}\mathbb{P}D \rightarrow \mathbb{P}D$ is the monad multiplication μ ; `↓` : $D \rightarrow \mathbb{P}D$ is the unit.

So the Smyth power domain is treated categorically as a **monad**, not just a domain construction.

6. Concrete syntax endofunctors (Exercises 6.21–6.23)

`Exercise623.lean` is the big applied example:

- **GExpr** — syntactic functor expressions (`N`, `X`, `+`, `×`)
- **gFunctor** — every `GExpr` is an `Endofunctor` of the strict-map category `ScottSys`
- **TexpF** — the functor $T(X) = N \oplus ((X \times X) + (X \times X))$ from Exercise 6.23
- **Colimit tower** $\text{gColim } T = \bigcup_n T^n(\{\Gamma\})$ and the solution domain `Exp`

Related files: `Exercise617.lean`, `Exercise617Gen.lean`, `Exercise622.lean`, `Exercise621.lean`, `Exercise619PartB.lean`.

7. Other categorical uses

- **Bottom functor** `(·)_⊥` — Exercise 6.26 (`Exercise626.lean`): strict functor on Scott's category.
- **Functoriality of** `≤` — Exercise 8.19 (`Exercise819.lean`): covariant/contravariant functoriality of embedding in `prod`, `sum`, and `exp`.
- **Exercise 6.29 indexed products/sums** — `iproduct`, `iotimes` as categorical-style universal constructions over index types.
- **Definition 8.9 / Prop 8.10** — universal-domain combinators built via `prodMap`, `sumMap`, `expMap` functor laws.

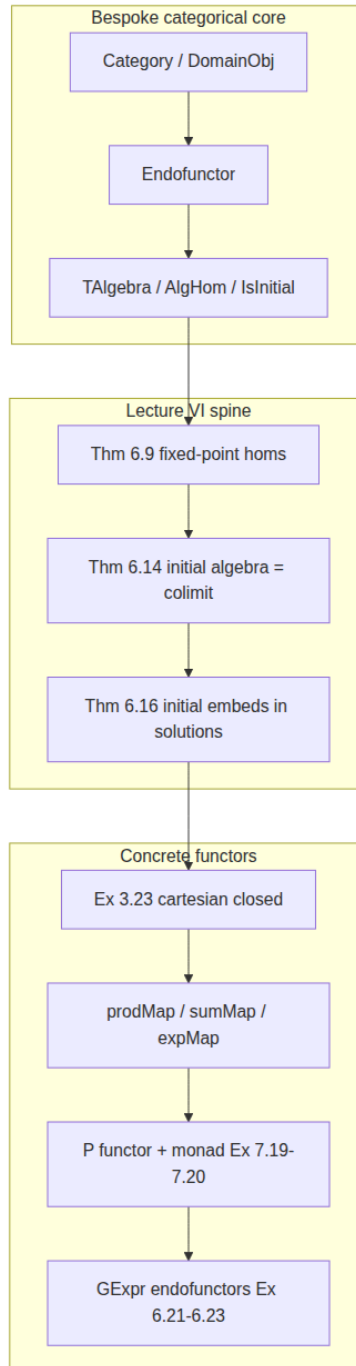
What is *not* category theory (but often adjacent)

Most of the repo uses **order theory**, not category theory:

- `Element`, `element_le`, `OrderIso`, `DomainIso`, `≤`, `<`
- Fixed-point theorems (Lecture IV) via approximable maps, not general limits/colimits

These are Scott's domain-theoretic tools; they overlap with categorical language (`Iso`, "universal") but are not built on the `Category` / `Endofunctor` API.

Summary map



Bottom line

Category theory here is concentrated in:

1. **Lecture VI** — domain-equation machinery (functors, algebras, initial algebras, colimits)
2. **Lecture III** — cartesian-closed structure (Exercise 3.23)
3. **Domain constructors** — functorial `×` / `+` / `→`
4. **Lecture VII** — power-domain monad (Exercises 7.19–7.20)
5. **Exercises 6.21–6.23** — `GExpr` syntax and concrete endofunctors

There is no broad category-theory library and no Mathlib `CategoryTheory` import; it is Scott's minimal categorical vocabulary, implemented choice-free for the flagship VI results.